Mercury × CrypTen: Reducing Communication Rounds for Privacy-Preserving ML Inference over PIE-Encoded Rationals

Author One and Author Two

Affiliation email@example.com

Abstract. *Briefly state the problem, the main technical idea, the protocol additions (FuzzyRound and comparison), the integration with CrypTen, and the main empirical result.*

Keywords: secure multi-party computation · privacy-preserving machine learning · CrypTen · secure inference

1 Introduction

Motivate privacy-preserving ML inference, explain why communication rounds are a bottleneck, introduce the mismatch between standard CrypTen-style arithmetic and Mercury/PIE arithmetic, and summarize the paper's main contributions and results.

1.1 Motivation

Explain why reducing round complexity matters in practical deployments, especially in LAN and WAN settings, and why this is important for private inference.

1.2 Problem Statement

State clearly what problem the paper addresses: integrating Mercury arithmetic into CrypTen while preserving support for inference workflows that require nonlinear operations such as ReLU and softmax-related steps.

1.3 Our Approach

Summarize the main idea of the paper: combine Mercury arithmetic with CrypTen, introduce FuzzyRound and a quantized comparison protocol, and evaluate the resulting system experimentally.

1.4 Contributions

List the main technical and empirical contributions of the paper in 4–5 concise bullets.

2 Background and Setting

Our work sits at the intersection of secure machine-learning inference and low-round MPC arithmetic. This section introduces the technical setting needed for the rest of the paper. We first review CrypTen as the baseline framework for MPC-based inference, then summarize the Mercury and PIE arithmetic model that motivates our integration. We next state the threat model and preprocessing assumptions used in our protocols and benchmarks, and finally clarify the scope of the workloads considered in this paper.

2.1 CrypTen and MPC-Based ML Inference

CrypTen is a framework for privacy-preserving machine learning based on secure additive MPC. It provides a tensor-oriented programming model that allows machine learning workloads to be expressed using abstractions similar to those of conventional deep learning frameworks, while executing the underlying computations on secret-shared data.

The relevant operations in this work include linear layers, tensor arithmetic, and nonlinear steps such as ReLU and softmax-related computations. The practical performance of these workloads depends not only on local computation, but also on the rounds of communication induced by the underlying MPC protocols. In particular, round complexity becomes increasingly important in LAN and WAN deployments where communications are slower.

CrypTen provides a useful execution environment and baseline for secure inference, but its standard arithmetic stack does not directly match the PIE-encoded rational setting used by Mercury[**?**]. As a result, integrating Mercury into CrypTen requires adapting the arithmetic layer and adding support for comparison-oriented functionality needed by nonlinear inference operations. This is the setting considered in the remainder of the paper.

2.2 Mercury and PIE-Encoded Arithmetic

Our work builds on Mercury[**?**], a family of constant-round MPC protocols by Harmon and Delavignette for exact arithmetic over encoded rational numbers. Rather than representing real values using the standard fixed-point embedding $x = 2^f \tilde{x} \bmod p$, Mercury works over the set of Farey rationals

$$F_N := \left\{ \frac{x}{y} : |x| \leq N,\ 0 < y \leq N,\ \gcd(x, y) = 1,\ \gcd(p, y) = 1 \right\},$$

where p is a prime and $N = N(p)$ is chosen so that these rationals can be encoded into $\mathbb{F}_p$. In the Mercury setting, a rational value $x/y \in F_N$ is first mapped to a field element using the PIE encoding function $\mathsf{Enc}\cdot$, and the resulting field element is then secret shared among the parties.

This encoding supports exact secure arithmetic on shared rational values. In particular, Mercury provides protocols for addition, subtraction, multiplication,

and division over encoded elements of F_N. Addition and subtraction are local and therefore have round complexity 0, multiplication requires one round, and division requires at most three rounds. Moreover, multiplication or division by a public integer can be performed locally as long as the resulting value remains in F_N. These properties make Mercury attractive for low-round arithmetic on rational-valued data.

At the same time, Mercury differs fundamentally from the arithmetic model typically used in MPC frameworks for machine learning. Standard MPC fixed-point systems represent a real value $\tilde{x}$ as a scaled integer in the field, typically $2^f \tilde{x} \bmod p$, and therefore require explicit truncation after multiplication to restore the original precision. Mercury instead performs exact arithmetic on PIE-encoded rationals. This is convenient for exact computation, but it does not by itself provide a direct analogue of the usual fixed-point normalization step required by ML workloads.

A key limitation is that F_N is not closed under addition and multiplication in general. In practice, one must choose a subset $G \subseteq F_N$ so that all intended intermediate values throughout the computation remain in F_N. This restricts circuit depth and becomes especially problematic for fixed-point-style workloads, since multiplication doubles the effective precision and division may produce denominators that are no longer powers of two. As observed in prior work [citation], Mercury is therefore only partly compatible with fixed-point arithmetic in its original form: addition and subtraction transfer directly, but multiplication and division require an additional mechanism to bring the result back to the target precision.

To address this issue, we introduce FuzzyRound, a probabilistic rounding protocol that approximates a shared rational value by one with denominator B^k. When composed with Mercury, FuzzyRound makes it possible to normalize products and quotients back to a desired precision and thereby obtain practical fixed-point arithmetic over PIE-encoded rationals. This mechanism is also central to our treatment of comparison-based operations later in the paper.

2.3 Threat Model and Assumptions

We work in the semi-honest (honest-but-curious) model. That is, all parties are assumed to follow the prescribed protocol, but an adversary controlling up to the tolerated threshold of parties may attempt to learn additional information from the messages and internal state observed during the execution. Our goal is therefore to protect the confidentiality of the parties' inputs and intermediate values against passive leakage, rather than to address malicious deviations from the protocol specification.

The underlying arithmetic protocols are instantiated over secret sharings in a prime field and are used to evaluate inference workloads on secret-shared data. We focus on the setting in which the parties jointly compute on PIE-encoded rational values using Mercury-backed arithmetic within the CrypTen execution model. As in the rest of the paper, we restrict attention to inference rather than training.

In addition to the computing parties, our implementation assumes access to a trusted third party (TTP) for preprocessing randomness. The TTP is used only to generate correlated random material needed by certain subprotocols, most notably random masks and their exact bit decompositions for the comparison routines described later in the paper. The TTP does not participate in the online inference phase, does not receive the parties' private inputs, and does not observe the opened intermediate values used during the online execution.

We therefore separate the protocol assumptions into two layers. The first layer is the standard semi-honest security assumption for the MPC parties. The second layer is a preprocessing assumption under which the TTP is trusted to generate correct and consistent randomness and to distribute it securely to the parties before the online phase begins. In particular, when we invoke ACCO-style masked comparison in our setting, we assume that the TTP provides both a random mask r and the exact bit decomposition of that same r. This replaces ACCO's original bit-decomposition step and is the reason our implementation does not need to rely on the Mersenne-prime decomposition mechanism of the original construction.

This model is appropriate for our benchmarking study because it cleanly separates online communication costs from preprocessing costs and allows us to focus on the interaction between Mercury-backed arithmetic, quantized comparison, and the CrypTen inference pipeline. At the same time, it should be kept in mind that the resulting performance numbers are obtained under a preprocessing-assisted threat model and should be interpreted accordingly.

Finally, we assume that all protocol parameters are chosen so that the relevant values remain within the admissible range of the Mercury/PIE representation throughout the computation. This includes the range constraints needed for arithmetic correctness, for the quantization step implemented by FuzzyRound, and for the integer comparison subroutines applied after scaling.

2.4 Problem Scope

Clarify that the focus of the paper, and specify which operations or model classes are in scope.

3 System Overview

Present the overall design of the Mercury × CrypTen system and explain how the components fit together at a high level.

3.1 Design Goals

Explain the main design goals, such as reducing rounds, preserving compatibility with CrypTen abstractions, and supporting practical inference workflows.

3.2 Architecture of Mercury ×rypTen

Describe how Mercury-backed arithmetic is integrated into the CrypTen stack and where the new protocol components fit into the architecture.

3.3 Execution Flow

Walk through the high-level execution path from encoded inputs to final model outputs, emphasizing where Mercury arithmetic and comparison are invoked.

4 Mercury-Backed Arithmetic in CrypTen

This section describes how Mercury-backed arithmetic is integrated into the CrypTen execution model. At a high level, our goal is not to replace CrypTen as an inference framework, but to replace or augment the underlying arithmetic layer used for secure computation. In particular, we use Mercury protocols to support secure computation over PIE-encoded rationals while retaining CrypTen's tensor-oriented programming model and distributed inference workflow [**?**,**?**].

This integration is motivated by the observation that the arithmetic core of secure inference has a direct effect on both communication rounds and end-to-end latency, especially in distributed deployments. CrypTen provides a practical baseline for MPC-based inference, but its standard fixed-point arithmetic stack is not naturally aligned with the Mercury/PIE representation [**?**,**?**,**?**]. Our approach is therefore to preserve the higher-level inference interface while substituting Mercury-backed protocols where the representation and round behavior are advantageous.

The resulting system is most natural for linear inference workloads. Addition, subtraction, multiplication, and division can all be expressed within the Mercury arithmetic model, but nonlinear inference requires additional support beyond the arithmetic layer alone. For this reason, we first describe the supported arithmetic operations in the integrated system and then turn in Section 5 to the extra protocols needed for nonlinear inference.

4.1 Supported Operations

The Mercury-backed arithmetic layer supports the core operations needed for secure linear inference: addition, subtraction, multiplication, and division over PIE-encoded rational values [**?**]. These operations are applied to secret-shared encodings of elements in F_N, and, as discussed in Section 2, the resulting protocols differ from standard fixed-point MPC in both representation and normalization behavior.

Addition and subtraction are local operations in Mercury and therefore require no communication rounds [**?**]. This makes them natural building blocks for affine transformations and tensor updates within inference workloads. Multiplication is supported as a constant-round protocol, and division is also supported

directly, which is one of the main conceptual differences between Mercury and standard fixed-point MPC stacks [**?**]. Moreover, multiplication by a public scalar is local, which is particularly useful when rescaling rounded values or applying known constants inside model computations [**?**].

From the perspective of CrypTen integration, these operations form the arithmetic basis for linear layers and other tensor computations that do not require comparison. In particular, secure matrix multiplication and linear transformations can be built from repeated additions and multiplications, while bias additions and public rescalings remain cheap. This makes Mercury-backed arithmetic especially appealing for the linear portions of inference, where the dominant cost is often the interaction pattern induced by repeated multiplications across the network [**?**,**?**].

At the same time, the supported operations must be interpreted together with the representation constraints of Mercury. The arithmetic is exact over PIE-encoded rationals as long as intermediate values remain within the admissible range of the encoding domain [**?**]. Thus, while the supported operation set is rich, practical usage still depends on controlling precision growth and denominator structure throughout the computation. This is one reason why multiplication and division, although directly supported, often need to be paired with a normalization step such as FuzzyRound before their outputs are fed into later stages of an inference pipeline.

In summary, the Mercury-backed layer directly supports the arithmetic operations required for linear secure inference and provides a cleaner rational-arithmetic model than conventional fixed-point encodings. However, this arithmetic support alone is not sufficient for nonlinear inference, which motivates the additional protocols introduced in Section 5.

4.2 Integration into the CrypTen Runtime

Explain how these operations are exposed within CrypTen and how tensors or operators are mapped to Mercury-backed implementations.

4.3 Communication and Round Behavior

Analyze where the round savings come from and how the Mercury-backed execution differs from standard CrypTen in communication structure.

4.4 Practical Constraints

Discuss limitations and assumptions related to encoding, precision, range, and compatibility with higher-level inference tasks.

5 Enabling Protocols for Nonlinear Inference

$a = c \pmod 2$ The arithmetic protocols of Mercury provide efficient secure support for addition, subtraction, multiplication, and division over PIE-encoded

rationals [?]. These operations are sufficient for linear inference, but they do not by themselves support the nonlinear components that appear in many practical inference pipelines [?,?]. In particular, operations such as ReLU, max-like computations, and parts of softmax-related workflows require comparison functionality [?,?], whereas Mercury natively operates on encoded rational values rather than on standard fixed-precision integers [?].

This mismatch creates two technical challenges. First, comparison-based operations require a notion of sign or ordering, but PIE-encoded rationals are not directly compared in the same way as conventional fixed-point field encodings [?,?]. Second, after multiplication and division, Mercury values may no longer lie in the representation best suited for comparison or for subsequent fixed-precision computation [?]. To address these issues, we introduce two additional protocol components. The first is FuzzyRound, which probabilistically rounds a shared rational to a nearby value with denominator B^k. The second is a quantized comparison procedure that applies integer comparison to the result of this rounding step.

Together, these protocols extend Mercury × CrypTen beyond purely linear computation and make it possible to support comparison-based nonlinear inference operations while remaining within the Mercury/PIE arithmetic setting.

5.1 Why Comparison Is Needed

Comparison is a basic primitive for several nonlinear operations used in machine-learning inference. The most direct example is ReLU, which requires determining whether an input is positive and then conditionally retaining or zeroing that value [?]. More generally, max-like operations, thresholding steps, and parts of softmax-related workflows also require the ability to distinguish positive from non-positive values or to compare one secret value against another [?,?].

In conventional MPC frameworks, such operations are typically built on top of fixed-point encodings together with comparison protocols tailored to integer or bit-level representations [?,?]. In our setting, however, values are represented as PIE-encoded rationals and manipulated using Mercury arithmetic [?]. As a result, comparison cannot simply be treated as a direct extension of the arithmetic layer. Instead, it must be mediated through a representation that is both compatible with Mercury and suitable for secure integer comparison.

Our approach is to reduce comparison on PIE-encoded rationals to comparison on shared integers. To do so, we first use FuzzyRound to map a shared rational to a nearby value with a fixed denominator B^k. We then multiply by the public value B^k to obtain a shared integer and apply an integer comparison protocol to that result. This yields a probabilistic quantized comparison procedure that is sufficient for the nonlinear inference operations considered in this paper.

This design reflects a practical tradeoff. Rather than attempting to define a direct comparison protocol over general PIE-encoded rationals, we compare quantized representatives of those values. This keeps the comparison layer compatible with the surrounding Mercury arithmetic, keeps the online cost low,

and provides a workable path to supporting nonlinear inference in Mercury × CrypTen.

5.2 FuzzyRound

A central challenge in combining Mercury arithmetic with machine-learning inference is that Mercury operates on PIE-encoded rationals, whereas inference workloads are typically organized around a fixed working precision. In particular, after multiplication or division, the result may no longer have the denominator or scale expected by later stages of the computation. To address this issue, we introduce FuzzyRound, a probabilistic rounding protocol that maps a shared rational value to a nearby value with denominator B^k.

Let $x/y \in F_N$, and let the parties hold a sharing of $\mathsf{Enc}x/y$. Given a base B, a precision parameter k, and a statistical security parameter σ, FuzzyRound outputs a sharing of a value of the form

$$\frac{z}{B^k} = \frac{\lfloor xB^k/y \rfloor}{B^k} + \frac{b}{B^k}, \qquad b \in \{0,1\},$$

so that the output denominator is exactly B^k.

At a high level, FuzzyRound first scales the shared value by the public factor B^k, then masks the scaled value using random integer and fractional offsets, reconstructs the masked quantity, and finally removes the mask after determining whether the output should be rounded up or down. The role of the mask is to hide the underlying secret during reconstruction, while still allowing the parties to recover a quantized output with the desired denominator.

The protocol is probabilistic only in the final rounding direction. If we write

$$\text{frac} = \frac{xB^k}{y} - \left\lfloor \frac{xB^k}{y} \right\rfloor \in [0,1),$$

then FuzzyRound rounds down with probability $1 - \text{frac}$ and rounds up with probability frac. Thus, the output is always one of the two adjacent grid points with denominator B^k.

For our purposes, the most important guarantee is that the output stays within one quantization step of the input. More precisely, if

$$[z/B^k] \leftarrow \textsc{FuzzyRound}([x/y], B, k, \sigma),$$

then

$$\left| \frac{x}{y} - \frac{z}{B^k} \right| < \frac{1}{B^k}.$$

In particular, if $x = 0$, then the protocol outputs $z = 0$. This makes FuzzyRound suitable as a normalization step after Mercury operations: it restores the target denominator needed by later fixed-precision computations while preserving a controlled approximation to the original rational value.

In the comparison workflow used in this paper, FuzzyRound serves a second purpose. After rounding to denominator B^k, we can multiply locally by the public value B^k and obtain a shared integer z. Because the PIE encoding agrees with the standard signed embedding for integers in the relevant range, this integer can then be passed to an integer comparison subroutine. In this way, FuzzyRound acts as the bridge between Mercury's rational arithmetic and the integer-based comparison machinery required by nonlinear inference operations.

The online cost of FuzzyRound is low. Its main online interaction is the reconstruction of the masked value; the remaining steps consist of local additions, multiplication by public constants, and local post-processing on the reconstructed quantity. This makes FuzzyRound a practical normalization primitive for the benchmarking setting considered here.

Finally, FuzzyRound is subject to parameter constraints that ensure both correctness and security. In particular, the scaled value xB^k/y must remain in the supported range, and the masking terms must be chosen so that no overflow occurs during the computation. In our benchmarks, we select parameters conservatively so that the protocol operates within the admissible range of the Mercury representation.

5.3 Probabilistic Quantized Comparison to Zero

To support comparison-based nonlinear inference steps, we convert a PIE-encoded rational into a shared integer and then invoke an integer comparison routine. The key idea is to combine FuzzyRound with a comparison protocol that operates on standard integer sharings.

Let $x/y \in F_N$, and let the parties hold a sharing of $\mathsf{Enc}(x/y)$. For notational simplicity, we write $[x/y]$ in place of $[\mathsf{Enc}(x/y)]$. We define the probabilistic quantized comparison protocol

$$[\beta] \leftarrow \textsc{Sign}_{B,k}([x/y])$$

as follows:

1. compute
$$[z/B^k] \leftarrow \textsc{FuzzyRound}([x/y], B, k, \kappa);$$
2. scale locally by the public value B^k to obtain
$$[z] = B^k \cdot [z/B^k];$$
3. invoke an integer comparison routine on $[z]$ and 0 to obtain
$$[\beta] \leftarrow \textsc{CompareZero}([z]).$$

For integers in the relevant range, the PIE encoding agrees with the usual signed embedding modulo p, so the output of the scaling step can be treated as a standard shared integer. The resulting protocol is probabilistic near zero

because FuzzyRound rounds up or down according to the fractional part of xB^k/y, with error strictly less than $1/B^k$. In particular, if

$$\frac{z}{B^k} = \frac{\lfloor xB^k/y \rfloor}{B^k} + \frac{b}{B^k}, \qquad b \in \{0, 1\},$$

then $\beta = 1$ if and only if $z > 0$. Thus the protocol computes the sign of the quantized value produced by FuzzyRound, rather than an exact comparison of x/y with 0.

5.4 Instantiating Integer Comparison

In our implementation, we instantiate CompareZero using an ACCO-style masked comparison protocol. The original ACCO construction uses a Mersenne prime field to derive a valid bit decomposition of a random masking value from random bits. In our setting, however, a trusted third party provides both a random mask r and the exact bit decomposition of that same r during preprocessing. As a result, we bypass ACCO's original Mersenne-prime-based decomposition step and use ACCO as a masked comparison subroutine on the post-rounded integer.

Given a shared integer $[z]$, the parties compute a masked value $c = u + r \bmod p$, where $u = 2z$, open c, and then combine the public least significant bit of c with the secret-shared least significant bit of r and a bitwise comparison result indicating whether $r < c$. This yields a shared bit encoding whether $z < 0$ or, equivalently after sign adjustment, whether $z > 0$. In our setting, the expensive part of ACCO's original decomposition step is replaced by trusted preprocessing of masks and mask bits.

5.5 Correctness and Security Sketch

Correctness follows from two observations. First, FuzzyRound outputs a value of the form

$$z/B^k = \lfloor xB^k/y \rfloor / B^k + b/B^k, \qquad b \in \{0, 1\},$$

with error strictly less than $1/B^k$. Second, for integers in the relevant range, the PIE encoding coincides with the standard signed embedding modulo p, so the scaled value $[z]$ may be treated as an ordinary shared integer. The comparison stage therefore returns the sign of the quantized value z.

Security follows by composition from the security of FuzzyRound, local public scaling, and the security of the ACCO-style masked comparison instantiated with trusted preprocessing. We defer the full details to the appendix.

6 Implementation

Describe the implementation details needed to understand the evaluation and reproduce the system behavior.

6.1 Codebase and Integration Details

State which codebase was modified, what components were added or replaced, and how the Mercury and CrypTen pieces were connected in practice.

6.2 Preprocessing and Randomness

Explain how randomness is generated or supplied, including any trusted third party preprocessing used for masks or bit decompositions.

6.3 Supported Models and Operators

List the models, layers, or operators currently supported by the implementation and clarify any gaps in coverage.

6.4 Engineering Choices

Document practical implementation choices such as batching, serialization, communication setup, deployment scripts, and system configuration.

7 Experimental Methodology

Explain how the evaluation was designed so that the results are meaningful, fair, and reproducible.

7.1 Research Questions

State the concrete questions the experiments aim to answer, such as whether Mercury × CrypTen reduces rounds, improves runtime, and enables nonlinear inference operations.

7.2 Experimental Environments

Describe the deployment environments used for testing, such as same-machine, same-region, and different-region settings.

7.3 Metrics

Define the metrics collected in the experiments, including runtime, round complexity, communication volume, and any preprocessing costs if relevant.

7.4 Workloads

Describe the benchmark workloads, including arithmetic microbenchmarks and end-to-end inference tasks.

7.5 Fairness and Reproducibility

Explain how runs were scheduled and repeated, how time-of-day effects were controlled, and what steps were taken to make the experiments reproducible.

8 Evaluation

Present the main experimental results and explain what they show about the benefits and limits of the Mercury × CrypTen approach.

8.1 Arithmetic Microbenchmarks

Measure the cost of the core arithmetic operations in isolation and compare Mercury-backed behavior against the baseline.

8.2 End-to-End Inference Benchmarks

Evaluate the effect of the integration on complete inference tasks and report the main practical performance results.

8.3 Communication and Round Analysis

Separate round reductions from runtime improvements and explain how communication structure affects the observed results.

8.4 Impact of Quantized Comparison

Measure the cost and practical effect of the comparison protocol and explain what new inference functionality it enables.

8.5 Sensitivity to Deployment Conditions

Analyze how the results vary across same-host, LAN, and WAN-style conditions.

8.6 Discussion of Bottlenecks

Identify the main remaining bottlenecks in the system and explain which parts still limit performance.

9 Discussion

Interpret the results at a higher level, explain where the approach helps most, and be honest about current limitations.

9.1 What the Integration Buys

Summarize the practical benefits of combining Mercury with CrypTen and identify the settings where the integration is most useful.

9.2 Limitations

Discuss important limitations, such as restricted operator support, quantized comparison behavior near zero, inference-only scope, and any trusted preprocessing assumptions.

9.3 When Mercury × CrypTen Is a Good Fit

Explain the kinds of workloads or deployment settings where this approach is likely to be most beneficial.

10 Related Work

Position the paper relative to prior work on private ML frameworks, MPC arithmetic, secure comparison, and privacy-preserving inference systems.

10.1 MPC Frameworks for ML

Discuss prior frameworks for MPC-based machine learning and explain how your work differs from or builds on them.

10.2 Fixed-Point and Rational MPC Arithmetic

Review the relevant work on fixed-point, rational, or encoded arithmetic in MPC and explain where Mercury and PIE fit in this landscape.

10.3 Secure Comparison for MPC

Discuss prior comparison protocols and explain how your comparison construction relates to them.

10.4 Privacy-Preserving ML Inference Systems

Compare the paper to other systems for secure inference and explain the practical niche of Mercury × CrypTen.

11 Conclusion

Briefly restate the problem, summarize the main contributions and empirical findings, and mention the most important next steps.

A Protocol Details

Provide full protocol listings or lower-level descriptions that are useful for completeness but too detailed for the main paper.

B Additional Proofs

Include full proofs of correctness and security for FuzzyRound, the comparison protocol, or any other supporting technical statements.

C Additional Experimental Results

Include extra plots, ablations, detailed tables, and robustness checks that support the main evaluation.